📘 **Theory of Computation (TOC) – Minor-2 Notes USICT GGSIPU | Exam Date: 01-05-2025**

---

1. Turing Machines

- **Definition**: A Turing Machine (TM) is a 7-tuple (Q, Sigma, Gamma, delta, q0, q_accept, q_reject).
- **Design**: Infinite tape, read/write head, finite control.
- **Example**: TM for L = {a^n b^n | n ≥ 1}.
- **Extensions**:
  - Multi-tape TM
  - Non-deterministic TM
  - Stay-put TM
- **Church–Turing Thesis**: All algorithmically solvable problems can be solved by TM.

---

2. Recursive and Recursively Enumerable Languages

- **Recursive (Decidable)**: TM halts for all inputs.
- **Recursively Enumerable (RE)**: TM halts for valid inputs; may loop on invalid ones.
- **Closure Properties**:
  - Recursive: Union, Intersection, Complement
  - RE: Union, Intersection (not Complement)

---

3. Halting Problem

- **Definition**: Whether a TM halts on a given input.
- **Result**: Undecidable using contradiction (diagonalization).

---

4. Rice's Theorem & Recursion Theorem

- **Rice's Theorem**: All non-trivial properties of TM languages are undecidable.
- **Recursion Theorem**: TM can refer to its own description (used in self-replicating code/quines).

---

5. Complexity Classes

- **P**: Problems solvable in polynomial time.
- **NP**: Problems verifiable in polynomial time.
- **co-NP**: Complements of NP problems.
- **PSPACE**: Polynomial space solvable problems.
- **NPSPACE**: Equal to PSPACE (Savitch's Theorem).

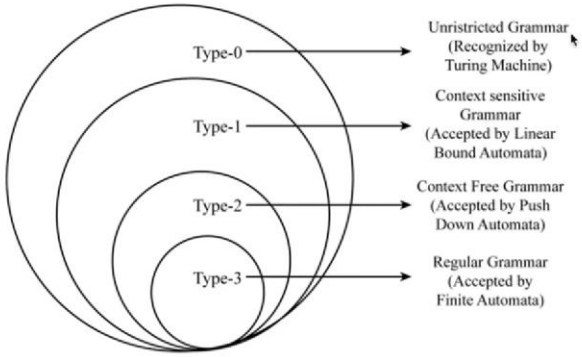
---

6. Polynomial Time Reductions, NP-Completeness & NP-Hardness

- **Reduction (≤p)**: Solving one problem using another in polynomial time.
- **NP-Complete**: In NP + every NP problem reduces to it.
  - Examples: SAT, 3-SAT, Clique, Vertex Cover.
- **NP-Hard**: As hard as NP-complete; not necessarily in NP.
  - Examples: Halting Problem, Tiling Problem.

Notes-YouTube

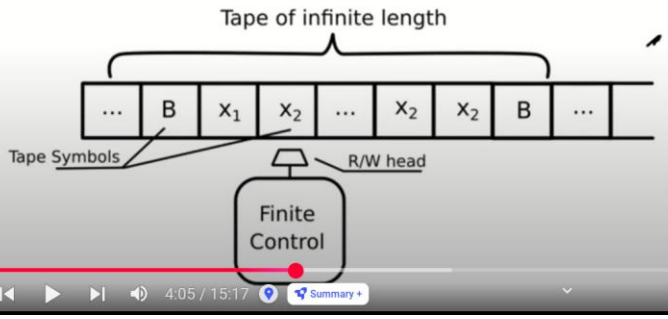
Introduction to Turing Machine

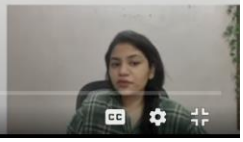


INTRODUCTION TO TURING MACHINE | TAFL | TOC | Automata Theory | UNIT 5 #tafl #toc #automatatheory

TMs are able to do any of the following Operations on Tape

1) Read a symbol from the tape.
2) Write a symbol to the tape.
3) Move the tape head, one step LEFT.
4) Move the tape head, one step RIGHT.

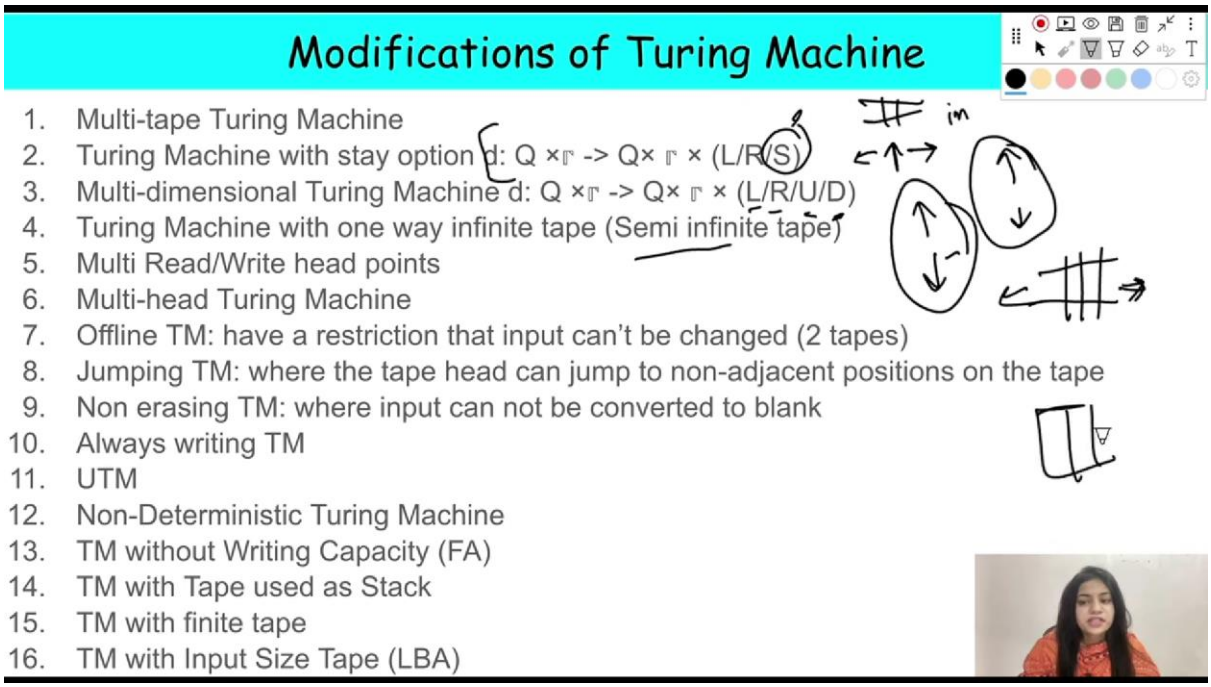




4:05 / 15:17 Summary +

Q: Is the Turing Machine tape infinite in both directions?
A: No, the standard Turing Machine tape is **infinite only in one direction** (usually to the right). However, there is a **variant** of the Turing Machine with a **doubly infinite tape** (infinite in both directions), but it is **not more powerful** than the standard one in terms of computational power.

Q: What does the Church–Turing Thesis state?
A: The Church–Turing Thesis states that any function that is computable by an algorithm can also be computed by a Turing Machine.



🧠 **Q1: What is a Deterministic Turing Machine (DTM)?**

A: A DTM is a Turing Machine where **for each state and input symbol, there is exactly one transition**.

➡️ **One path of execution** only.
➡️ Works in a **predictable and fixed** manner.

🤖 **Q2: What is a Non-Deterministic Turing Machine (NDTM)?**

A: An NDTM is a Turing Machine where **for some state and input symbol, there may be multiple possible transitions**.

➡️ Can explore **many computation paths in parallel** (theoretically).
➡️ Accepts input **if any path leads to acceptance**.

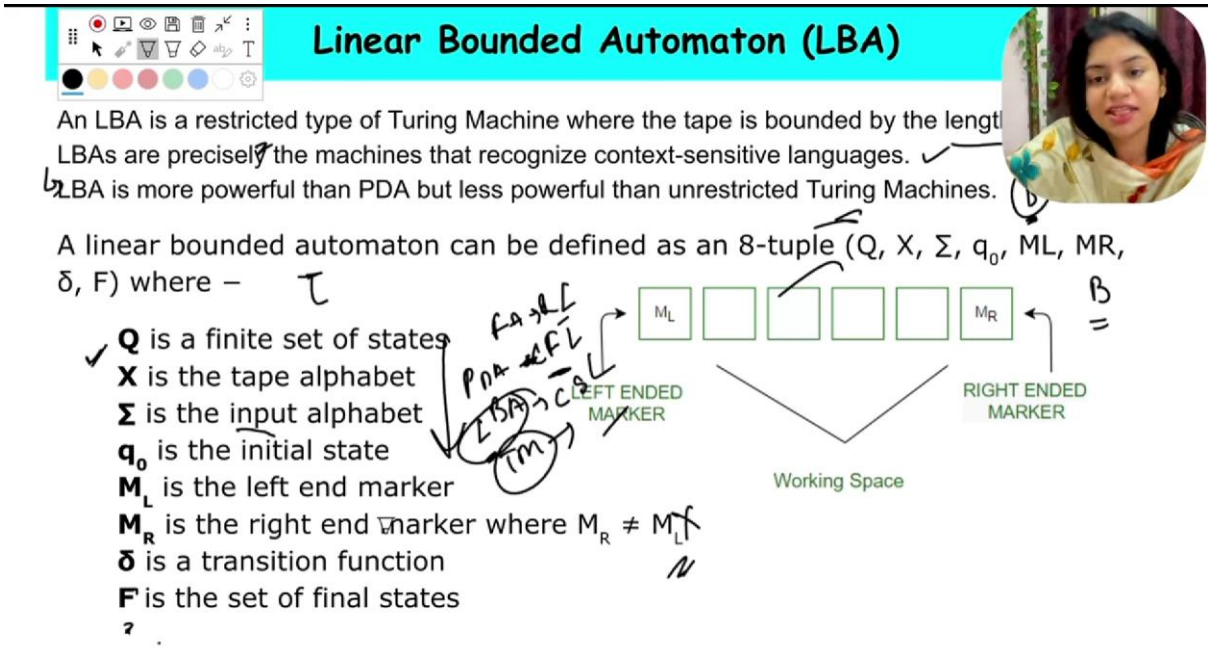
📌 Note: **NDTM and DTM have the same power** (they accept the same class of languages).

🌐 **Q3: What is a Universal Turing Machine (UTM)?**

A: A Universal Turing Machine is a Turing Machine that can **simulate any other Turing Machine**.

➡️ It takes as input the **description of another Turing Machine** and an **input string**, and simulates the behavior of that machine on that input.

➡️ **Key idea behind modern computers** — they can run any algorithm if programmed correctly.



USICT GGSIPU ke syllabus ke hisaab se (jo tune diya hai), usme **"Universal Turing Machine (UTM)" directly mentioned nahi hai**.

📌 **Kya Universal TM padhna zaruri hai?**

Answer: **Directly nahi, lekin indirectly useful ho sakta hai** — kyunki:

1. **Halting Problem** aur **Rice's Theorem** me UTM ka concept background me use hota hai.
2. Agar **theoretical MCQ ya assertion-reason** type questions aaye, toh **UTM ka basic idea helpful** hoga.

🔍 **Toh bas itna padhna kaafi hoga UTM ke liye:**

☑ Universal Turing Machine (UTM) – Basic Idea

- **Definition**:
  A Universal Turing Machine is a TM that can simulate **any other TM** on arbitrary input.

- **Motive**:
  Just like a **general-purpose computer**, UTM can take as input:
    - Description of a Turing Machine M
    - Input w to that TM

And simulate the behavior of M on w.

- **Symbolically**:

$$U(\langle M, w \rangle) = M(w)$$

- **Why Important?**
    - Proves that **a single TM** can be programmed to perform **any computation** — just like today's computers.
    - Core idea behind **Halting Problem** proof.

📚 **Exam Relevance:**

Topic	UTM Usefulness
Turing Machines	Not directly
Halting Problem	🔥 Yes, used in proof
Rice's Theorem	🔥 Yes, indirectly
Recursion Theorem	Slightly
Complexity Classes	❌ Not used

🧠 **So Final Verdict:**

UTM ka detailed design ya working yaad karne ki zarurat nahi hai, bas itna concept yaad rakho:
"Universal TM ek aisi machine hai jo kisi bhi Turing Machine ko simulate kar sakti hai."

Bilkul bhai! **Halting Problem** ka proof **important hai** — aur haan, **exam me aa sakta hai** in these forms:

- "Explain Halting Problem with proof."
- "Prove that the Halting Problem is undecidable."
- "Is Halting Problem recursively enumerable?"

So haan, **ye zarur padhna chahiye**.

☑ Halting Problem – Full Proof (with Explanation)

📚 **Precise Proof (Same as Uploaded Solution):**

Halting Problem:
Given a Turing Machine M and input w, decide whether M halts on w.

Language:

$$\text{HALT} = \{(M, w) \mid M \text{ halts on } w\}$$

Proof:

Assume: There exists a Turing Machine HHH which decides HALT.

- Input to H: (M, w)
- Output:
    - YES → if M halts on w
    - NO → if M does not halt on w

Construct a Machine H' using H:

- Input: (M, w)
- Working:
    - If H outputs YES → **Loop forever**
    - If H outputs NO → **Halt immediately**

*(Thus, H'' does the **opposite behaviour** of H.)*

Now Construct Machine H cap:

- Input: M
- Working:
    - Run H' on input (M, M)

(i.e., Machine MMM with its own description as input.)

Now run H^ on input H^ itself:

- Input: H^
- Working:
    - H’ will be given (H^,H^)
    - Two cases arise:
        - If H^ halts on H^→ H ’ will loop forever → contradiction.
        - If H^ does not halt on H^ → H ’ halts → contradiction.

Thus, **contradiction occurs** in both cases.

Conclusion:

Hence, no such machine HHH can exist.
Therefore, the Halting Problem is Undecidable.

🧠 **Final Mapping:**

Machine	**Meaning**
H	Assumed decider for HALT
H’	Modified machine (opposite behaviour of H)
H^	Final machine to create contradiction

✅ Proof is exactly according to your uploaded solution.
✅ Short, exam-ready, same logic flow (H → H' → Ĥ).
✅ Real standard explanation.

📝 **Short Answer (1-2 lines):**

The Halting Problem is undecidable.
This is proven by contradiction, assuming a decider H exists, constructing D that leads to paradox on input D(D).